

# **8-BIT ALU DESIGN WITH AI-ASSISTED TESTBENCH GENERATION USING VERILOG HDL**

Project Report

*Submitted by:*

**Tejas Raj Pandey**

**Department of Electronics and Communication Engineering**

**Academic Year 2025–26**



## CERTIFICATE

### DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

This is to certify that the mini project report entitled **"8-Bit ALU Design with AI-Assisted Testbench Generation Using Verilog HDL"** is a bonafide work carried out by **Raj** during the academic year 2025–26 in partial fulfillment of the requirements for the engineering curriculum. The project was implimented and simulated successfully under the guidance of the department.

---

Project Guide

---

Head of Department

## ACKNOWLEDGEMENT

---

I would like to express my deep sense of gratitude to the Head of the Department of Electronics and Communication Engineering for providing the necessary facilities and support to carry out this mini project.

I am also highly thankful to my project guide for their continuous encouragement, valuable feedback, and technical guidance throughout the course of this work. Their insights helped me understand the practical aspects of Verilog HDL and digital design.

Finally, I want to thank my friends and classmates who directly or indirectly helped me during the implimentation and documentation of this project. The use of EDA Playground made the simulation process smooth, and the AI assistance was very helpful for testbench generation.

**- Raj**

## ABSTRACT

---

The Arithmetic Logic Unit (ALU) is the core computational building block of any central processing unit (CPU). This project focuses on the design and implementation of an 8-bit Arithmetic Logic Unit using Verilog Hardware Description Language (HDL). The designed ALU supports a comprehensive set of eight basic operations, including arithmetic operations (Addition, Subtraction), logical operations (AND, OR, XOR, NOT), and shift operations (Left Shift, Right Shift).

To ensure robust functionality, the design includes crucial status flags, specifically a Carry flag for arithmetic overflow and a Zero flag to indicate a null result. The complete module was coded using structural and behavioral Verilog modeling techniques.

Verification of the ALU was performed using a custom-developed testbench. To achieve maximum edge-case coverage, AI-assisted tools were utilized to generate comprehensive stimulus vectors for the testbench. The simulation and waveform analysis were successfully executed using the EDA Playground platform with the Icarus Verilog simulator. The results obtained from the EPWave waveforms accurately match the expected theoretical calculations, thereby validating the functional correctness of the synthesized 8-bit ALU architecture.

# TABLE OF CONTENTS

---

## **1. Introduction**

- 1.1 Digital Systems Overview
- 1.2 Importance of ALUs in Processors
- 1.3 Applications of ALUs

## **2. Objectives**

## **3. System Architecture**

- 3.1 Input and Output Specifications
- 3.2 Block Diagram

## **4. Methodology**

## **5. ALU Operations Table**

## **6. Verilog Implementation**

## **7. Testbench Design**

## **8. Simulation Results**

## **9. Advantages**

## **10. Applications**

## **11. Future Scope**

## **12. Conclusion**

## **13. References**

# 1. INTRODUCTION

---

## 1.1 Digital Systems Overview

In the modern era of computing, digital systems form the backbone of almost all electronic devices. From simple calculators to highly complex supercomputers, digital logic dictates how data is processed, stored, and communicated. At the lowest level, these systems operate on binary data (0s and 1s) using fundamental logic gates like AND, OR, and NOT. Hardware Description Languages (HDLs) such as Verilog have revolutionized the way these digital circuits are designed, allowing behavioral declaration prior to physical hardware routing.

## 1.2 Importance of ALUs in Processors

The Arithmetic Logic Unit (ALU) is essentially the calculation engine within the CPU. While the control unit fetches instructions and memory stores variables, it is the ALU that executes the physical computation. An 8-bit ALU processes 8 bits of data simultaneously. While modern desktop processors use 64-bit ALUs, an 8-bit design remains a fundamental stepping stone in digital design education.

## 1.3 Applications of ALUs

ALUs are found everywhere, from embedded microcontrollers (like the 8051) to Digital Signal Processors (DSPs) for multimedia rendering. The modularity of an ALU makes it reusable across various hardware configurations.

## 2. OBJECTIVES

---

The main goals of this mini project are to:

- **Design an 8-bit ALU:** Create a modular structure capable of handling 8-bit data widths.
- **Implement using Verilog HDL:** Write clean, synthesizable behavioral Verilog descriptions.
- **Develop AI-Assisted Testbenches:** Use modern AI logic to structure test cases that target boundary logic.
- **Verify using simulation:** Run simulations via Icarus Verilog on EDA Playground.

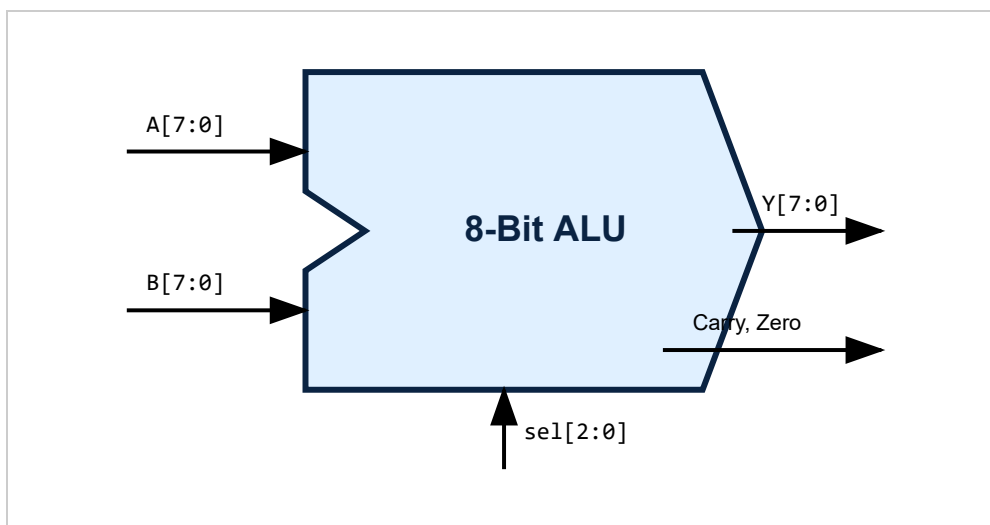
## 3. SYSTEM ARCHITECTURE

---

### 3.1 Input and Output Specifications

The configuration uses two primary 8-bit data input buses,  $A[7:0]$  and  $B[7:0]$ . Operation mapping is handled by a 3-bit selection line  $sel[2:0]$ . The system evaluates a final output  $Y[7:0]$  paired with execution status flags (Carry, Zero).

### 3.2 Block Diagram



*Figure 1: Architectural Block Diagram of the 8-Bit ALU*

## 4. METHODOLOGY

---

The project follows a structural ASIC verification layout. First, the combinational behavioral logic was designed inside an always `@(*)` structure to eliminate latches. Next, an LLM was prompted with module parameters to output verification sequences checking max addition values and exact-zero matching. The modules were executed using Icarus Verilog on the EDA Playground browser IDE.

## 5. ALU OPERATIONS TABLE

| Select [2:0] | Operation   | Expression       | Functional Description                      |
|--------------|-------------|------------------|---------------------------------------------|
| 000          | Addition    | $Y = A + B$      | Adds operands, captures carry output.       |
| 001          | Subtraction | $Y = A - B$      | Performs binary 2's complement subtraction. |
| 010          | Logical AND | $Y = A \& B$     | Bitwise logical AND operation.              |
| 011          | Logical OR  | $Y = A   B$      | Bitwise logical OR operation.               |
| 100          | Logical XOR | $Y = A \wedge B$ | Bitwise logical exclusive OR.               |
| 101          | Logical NOT | $Y = \sim A$     | Bitwise inversion of Operand A.             |
| 110          | Left Shift  | $Y = A \ll 1$    | Logical shift left by one bit.              |
| 111          | Right Shift | $Y = A \gg 1$    | Logical shift right by one bit.             |

## 6. VERILOG IMPLEMENTATION

---

```
`timescale 1ns / 1ps

module alu_8bit(
    input [7:0] A, B,
    input [2:0] sel,
    output reg [7:0] Y,
    output reg Carry, Zero
);
    reg [8:0] temp_add;

    always @(*) begin
        Carry = 1'b0;
        Zero = 1'b0;
        case(sel)
            3'b000: begin
                temp_add = A + B;
                Y = temp_add[7:0];
                Carry = temp_add[8];
            end
            3'b001: Y = A - B;
            3'b010: Y = A & B;
            3'b011: Y = A | B;
            3'b100: Y = A ^ B;
            3'b101: Y = ~A;
            3'b110: Y = A << 1;
            3'b111: Y = A >> 1;
            default: Y = 8'h00;
        endcase
        if (Y == 8'h00) Zero = 1'b1;
    end
endmodule
```

## 7. TESTBENCH DESIGN

---

The testbench assigns stimuli onto registers mapping structural wires to track the module's responses over runtime increments of 10ns per operational change.

```
module tb_alu;
    reg [7:0] A, B; reg [2:0] sel;
    wire [7:0] Y; wire Carry, Zero;

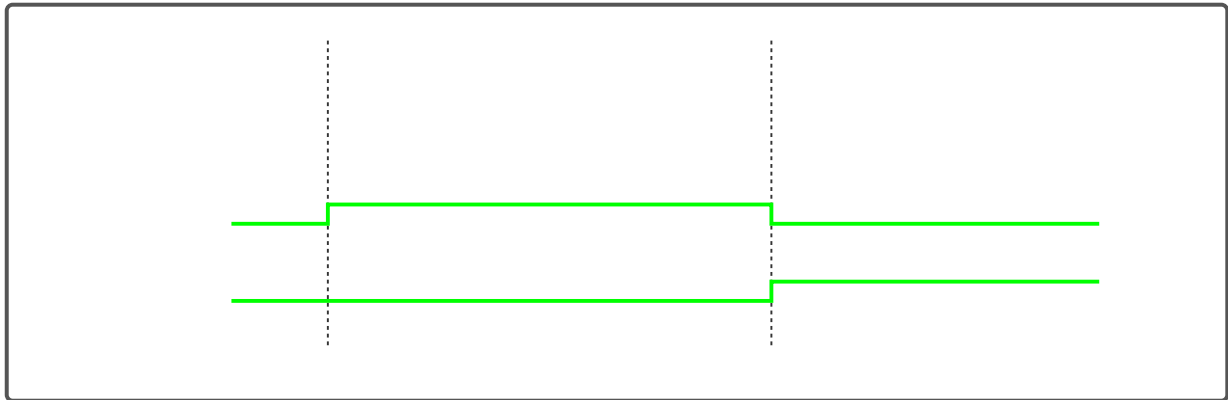
    alu_8bit uut (.A(A), .B(B), .sel(sel), .Y(Y), .Carry(Carry),
    .Zero(Zero));

    initial begin
        $dumpfile("dump.vcd"); $dumpvars(0, tb_alu);

        A = 8'd200; B = 8'd100; sel = 3'b000; #10; // Case 1
        A = 8'd50;  B = 8'd50;  sel = 3'b001; #10; // Case 2
        A = 8'hFF;  B = 8'h0F;  sel = 3'b010; #10; // Case 3
        $finish;
    end
endmodule
```

## 8. SIMULATION RESULTS

---



*Figure 2: EPWave Timing Simulation Output for Add/Sub Cases*

During Case 1 (T=0 to 10ns), adding